Lecture 5: Random Walks

A random walk models a path built from successive random steps, and this lecture uses the classic "drunkard's walk" to introduce both the mathematics of such processes and the simulation techniques needed to study them when analytic answers are out of reach.

Why random walks matter

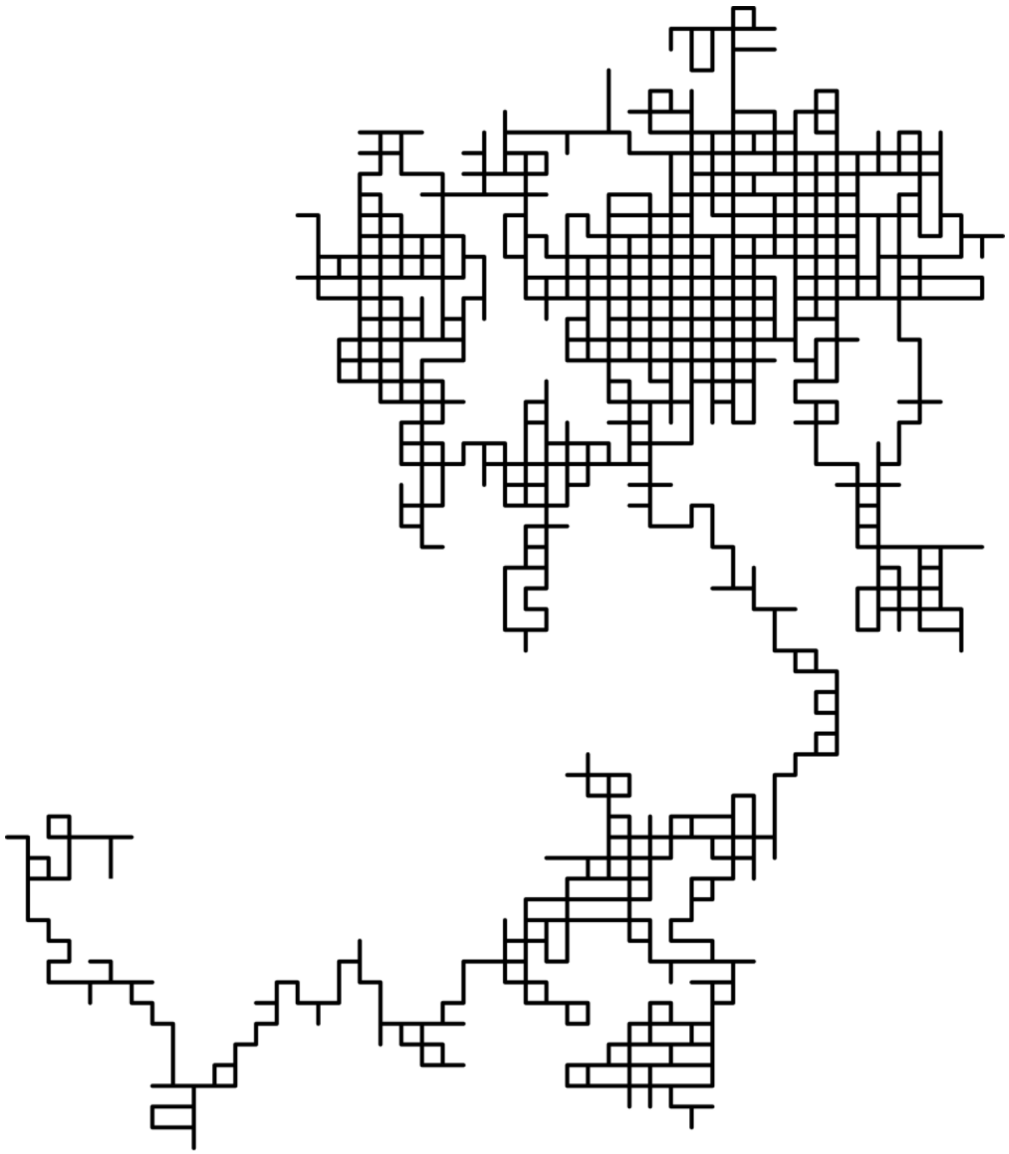
Formally, a **random walk** is a *stochastic process* describing a path consisting of a succession of random steps on some mathematical space. Despite sounding like a toy problem, the model appears across an enormous range of domains: the price of a fluctuating stock (the professor flags this one as debatable — whether stock prices truly behave like a random walk is an open empirical question), the path traced by a molecule traveling through a liquid or gas (**Brownian motion**), the search path of a foraging animal, the financial status of a gambler, and diffusion processes generally, like a drop of ink spreading through water. Applications span engineering, ecology, psychology, computer science, physics, chemistry, biology, economics, and sociology. The term itself was introduced by **Karl Pearson in 1905**.

Two features make this topic especially valuable pedagogically. First, realizations of random walks are obtained by **Monte Carlo simulation**: sometimes you cannot derive the answer analytically, but you can build a simulation, run it many times, and learn from the pattern of results. Second, implementing a walk forces engagement with reusable programming techniques — notably **classes** and **plotting**.

Defining the walk: taxonomy and the drunkard's walk

The most popular model places the walk on a **regular lattice**, where at each step the location jumps to another site according to some probability distribution. In a **simple random walk**, the location can only jump to *neighboring* sites of the lattice, forming a lattice path. When the probabilities of jumping to each immediate neighbor are identical, the walk is **simple symmetric**. The best-studied case is the walk on the d-dimensional integer lattice $\mathbb{Z}^d$ (the *hypercubic lattice*). If the state space is finite instead, the model becomes a **simple bordered symmetric random walk**: transition probabilities then depend on location, because margin and corner states restrict movement.

The drunkard's walk is exactly a simple symmetric random walk on the two-dimensional lattice: a drunkard starts at the origin (the center of a field drawn as a grid), and at each step picks north, south, east, or west with equal probability, moving one unit. One trial might send him south first; the red marker tracking his position simply moves with him. That is all a random walk is — from wherever you currently are, repeatedly take a step in a randomly chosen direction.



Source: Wikipedia, article "Random walk".

The elementary one-dimensional walk and counting outcomes

The canonical example lives on the integer number line $\mathbb{Z}$: start at 0, and at each step move $+1$ or -1 with equal probability. Picture a marker at zero and a fair coin: heads moves it one unit right, tails one unit left. After five flips the marker must sit on one of $-5, -3, -1, 1, 3, 5$ — note only odd positions are reachable, since five steps cannot produce an even net displacement. The outcome counts are far from uniform: there are 10 ways to land on 1 (three heads, two tails, in any order), 10 ways to land on -1, 5 ways each for ± 3, and just 1 way each for ± 5.

Formally, take independent random variables $Z_1, Z_2, \ldots$ where each is $+1$ or -1 with 50% probability, set $S_0 = 0$, and define

$$S_n = \sum_{j=1}^{n} Z_j.$$

The series $\{S_n\}$ is the **simple random walk on** $\mathbb{Z}$, and since each step has length one, S_n gives the net distance walked. Counting explains the outcome frequencies above: there are 2^n distinct n-step walks, all equally likely. For $S_n = k$, the number of $+1$'s must exceed the number of -1's by exactly k, so $+1$ must appear $(n + k)/2$ times among the n steps — hence the number of walks achieving $S_n = k$ equals the number of ways of choosing those positions, i.e. $\binom{n}{(n+k)/2}$. This is why extreme positions (± 5) are rare and central ones (± 1) are common, and why many of these results can also be derived from properties of Pascal's triangle.

Long-run behavior: zero drift, growing spread

Two exact expectations reveal why intuition about random walks is unreliable. First, the expected position is always zero:

$$E(S_n) = \sum_{j=1}^{n} E(Z_j) = 0,$$

by the finite additivity of expectation — the mean of all coin flips approaches zero as flips accumulate. So the walk has no tendency to drift anywhere. Yet the walker does not stay near the origin either. Using independence and the fact that $E(Z_n^2) = 1$, the cross terms $E(Z_i Z_j)$ for $i < j$ vanish, giving

$$E(S_n^2) = \sum_{i=1}^{n} E(Z_i^2) + 2 \sum_{1 \le i < j \le n} E(Z_i Z_j) = n.$$

This hints that the expected translation distance $E(|S_n|)$ grows on the order of $\sqrt{n}$ — and indeed,

$$\lim_{n \to \infty} \frac{E(|S_n|)}{\sqrt{n}} = \sqrt{\frac{2}{\pi}}.$$

The resolution of the apparent paradox: the walker's *average position* stays at zero, but his *typical distance from the start* grows like $\sqrt{n}$. This is precisely the kind of question ("after a hundred steps, how far is he likely to be from the start?") where intuition fails and simulation earns its keep.

Recurrence and the gambler's ruin

If the walk is allowed to continue forever, a striking result emerges: a simple random walk on $\mathbb{Z}$ will cross **every point an infinite number of times**. This goes by several names — the *level-crossing phenomenon*, *recurrence*, or the *gambler's ruin*. The last name comes from gambling: a gambler with finite money playing a fair game against a bank with infinite money will eventually lose, because his fortune performs a random walk that must reach zero at some point, ending the game. So the answer to "could he even end up where he began?" is not just yes — he returns to every point endlessly.

Hitting behavior can be quantified exactly. If a and b are positive integers, the expected number of steps until a one-dimensional walk starting at 0 first hits b or $-a$ is ab. The probability that it hits b before $-a$ is

$$\frac{a}{a+b},$$

which follows from the fact that the simple random walk is a **martingale**. These expectations and hitting probabilities can be computed in $O(a + b)$ time for general one-dimensional random-walk Markov chains.

Studying walks by simulation

Because quantities like the distribution of distance-after-n-steps resist casual derivation, the lecture's methodological thrust is simulation: encode the drunkard as a class holding his position, repeatedly draw a uniformly random direction, update the position, and plot the resulting trajectories. Running many trials produces pictures like scattered endpoint clouds (mostly blue dots with a few red), letting us empirically answer the questions simulation was chosen for — how far the walker gets, whether he wanders back, and whether he can end up where he began. The same machinery scales visually: as steps multiply and shrink, the walk's character changes qualitatively, approaching Brownian motion in the limit — the mathematical image of the diffusion processes that motivated the topic.

The structure of the simulation: one walk, many walks, an average

The plan for answering the motivating question is a three-step recipe: simulate **one** walk of k steps; then simulate n such walks; finally report the **average distance from origin** across those n walks. The middle step exists for a statistical reason: a single walk tells us almost nothing, because it is just one random outcome drawn from an enormous space of possibilities.

This recipe is a Monte Carlo construction — the standard way to get realizations of a random walk, per the mathematical literature. The object being simulated has a long pedigree: the term "random walk" was introduced by Karl Pearson in 1905, and in some texts the model is literally called a *drunkard's walk*, which is exactly the story this lecture tells. The canonical formal version places a marker at 0 on the number line $\mathbb{Z}$ and moves it $+1$ or -1 with equal probability each step: with independent variables $Z_j \in \{+1, -1\}$, the position after n steps is $S_n = \sum_{j=1}^{n} Z_j$.

Why can't one walk suffice? Because individual outcomes scatter widely. After only five coin flips the marker can end at $-5, -3, -1, 1, 3$, or 5, and the endpoints are reached by very different numbers of step sequences (10 ways to land on 1, 5 ways on 3, 1 way on 5, symmetrically for negatives). Averaging over many walks is what converts scattered single outcomes into a stable estimate of typical behavior. Theory predicts what that average should look like: the mean displacement is zero, $E(S_n) = \sum_{j=1}^{n} E(Z_j) = 0$, but the mean squared displacement grows linearly, $E(S_n^2) = n$, so the expected distance from the origin scales like $\sqrt{n}$ — in fact,

$$\lim_{n\to\infty} \frac{E(|S_n|)}{\sqrt{n}} = \sqrt{\frac{2}{\pi}}.$$

This $\sqrt{n}$ growth of typical distance is precisely the kind of result the simulate-and-average procedure is built to expose empirically.

Designing the abstractions before writing any code

Before coding, decide what the pieces are. Three abstractions are needed, and they map directly onto the story:

- **Location** — simply a place.
- **Field** — a collection of places and drunks.
- **Drunk** — somebody who wanders from place to place in a field.

The crucial engineering decision is *separation of concerns*: the Drunk does not need to know anything about the Field, and the Field does not need to know how the Drunk decides where to step next. Keeping the abstractions separate is what will allow different kinds of drunks to be swapped in later without rewriting everything. This is exactly the flexibility inheritance is designed to provide — the ability to "reuse code and to independently extend original software," with a superclass supplying "common interface and foundational functionality, which specialized subclasses can inherit, modify, and supplement."

Location: an immutable value type

The simplest piece is the `Location` class, a subclass of `object`. Its `__init__` takes an `x` and a `y` (both floats, per the docstring) and binds them to `self.x` and `self.y`. Its `move` method takes a `deltaX` and `deltaY` — also floats — and embodies the key design decision of the whole class:

```
def move(self, deltaX, deltaY):
    return Location(self.x + deltaX, self.y + deltaY)
```

`move` does **not** change this location; it *returns a new* `Location` whose coordinates are the old ones plus the deltas. That is why the slide flags this as an **immutable type**: a Location represents a fixed place, and "moving" means producing a brand-new Location rather than mutating one in place. Every step of a walk therefore creates a fresh value, never a side effect on a shared object.

Rounding out the class are the obvious accessors `getX` and `getY`, which simply return `self.x` and `self.y`, and a `__str__` method that renders the location as `'<'`, the string of `x`, `', '`, the string of `y`, `'>'`. Nothing deep — but readable printing makes debugging far more pleasant.

distFrom: the Pythagorean theorem at work

The second half of `Location` adds `distFrom(other)`, which computes the distance to another location in three lines: set `xDist = self.x - other.getX()`, set `yDist = self.y - other.getY()`, and return `(xDist**2 + yDist**2)**0.5`. As the professor emphasizes, nothing mysterious is happening — it is just the Pythagorean theorem, the Euclidean distance learned in high-school geometry.

The theorem states a fundamental relation in Euclidean geometry between the three sides of a right triangle: the area of the square on the hypotenuse equals the sum of the areas of the squares on the other two sides, written $a^2 + b^2 = c^2$. When Euclidean space is represented in a Cartesian coordinate system, Euclidean distance satisfies exactly this relation: the squared distance between two points equals the sum of squares of the differences in each coordinate. In `distFrom`, the horizontal leg of the triangle is `xDist`, the vertical leg is `yDist`, and the straight-line separation is the hypotenuse extracted by the `**0.5`. This little function supplies the very quantity the entire simulation reports — distance from the origin — so the accuracy of the whole experiment rests on this classical identity (one of the most-proved theorems in mathematics, by many geometric and algebraic methods).

Drunk: a base class designed to be inherited

The `Drunk` class is again a subclass of `object`. Its `__init__` takes a `name` that defaults to `None`, assumes the name is a string, and stores it as `self.name`. Its `__str__` returns the drunk's name when there is one, and otherwise returns `'Anonymous'` — so even an unnamed drunk prints as something sensible.

But the boxed warning on the slide is the real lesson: this class is **not intended to be useful on its own**. It is **a base class to be inherited from**. A generic `Drunk`, as written, does not know how to walk — no wandering behavior has been given to it at all. What it captures is what is *common to all drunks*: they have a name. The interesting behavior — *how* a particular drunk moves — will be supplied by subclasses that inherit from this base.

This is textbook inheritance in the sense the literature defines it: basing a class upon another class, retaining similar implementation, and deriving new subclasses from an existing superclass to form a hierarchy. In class-based object-oriented languages, a child class automatically inherits the instance variables and member functions of its superclass — here, every future kind of drunk gets `name`, `__init__`, and `__str__` for free — though constructors themselves are typically among the few things *not* inherited (along with destructors and overloaded operators, in languages like C++), which is why specialized subclasses define their own initialization. One distinction worth keeping in mind: inheritance proper is about reusing implementation, whereas *subtyping* establishes an is-a relationship; composition, by contrast, models a has-a relationship (a Field *has* drunks rather than being one). The mechanism has deep roots: the design traces to Ole-Johan Dahl and Kristen Nygaard's work culminating in Simula 67 — influenced by Tony Hoare's 1966 remarks on record

subclasses — and then spread through Smalltalk, C++, and Java to Python, whose `class Drunk(object)` syntax is a direct descendant.

Two Drunks, One Interface: Encoding Behavioral Bias

With `Location` and the abstract `Drunk` in place, the lecture introduces two subclasses that share an interface but differ in behavior — the whole point being to model different personalities and observe how personality changes the outcome of a walk.

- **`UsualDrunk`** implements `takeStep` by building a list of step choices — `(0,1)`, `(0,-1)`, `(1,0)`, `(-1,0)` — that is, north, south, east, west, each of unit length — and returning `random.choice(stepChoices)`. Every direction is equally likely every step: a *perfectly symmetric* random walk. This is exactly the textbook object from the Wikipedia material: a **simple random walk** on a lattice, where the walker "can only jump to neighboring sites," and in the *symmetric* variant each immediate neighbor gets the same probability.
- **`MasochistDrunk`** also chooses uniformly among four directions, but the step *sizes* are asymmetric: `(0.0, 1.1)` north, `(0.0, -0.9)` south, `(1.0, 0.0)` and `(-1.0, 0.0)` east/west. He does not refuse to go south — the bias is baked into the magnitudes. His expected vertical displacement per step is $\frac{1}{4}(1.1) + \frac{1}{4}(-0.9) = 0.05$ units northward, so on average he drifts north. This is also why his tuples contain floats while the usual drunk's contain integers.

Immutability as a Working Discipline

The lecture poses the same red question twice —

Square-root growth in the usual drunk's wanderings

Running `drunkTest` on `UsualDrunk` with walk lengths of 10, 100, 1000, and 10000 steps — 100 trials per length — produces these summary statistics for the final distance from the origin:

Steps	Mean	Max	Min
10	2.863	7.2	0.0
100	8.296	21.6	1.4
1000	27.297	66.3	4.2
10000	89.241	226.5	10.0

The striking pattern: **every time the number of steps is multiplied by ten, the mean distance rises by roughly a factor of three**. Since $\sqrt{10} \approx 3.16$, this is the signature of square-root growth. You can see it directly by dividing each mean by $\sqrt{\text{steps}}$: the ratio stays in a narrow band ($2.863/\sqrt{10} \approx 0.91$, $8.296/10 \approx 0.83$, $27.297/\sqrt{1000} \approx 0.86$, $89.241/100 \approx 0.89$). The drunk does wander away from the origin, but far more slowly than the step count increases. Note also the min of 0.0 at 10 steps: with so few trials steps, at least one walk happened to end exactly where it started — short walks are extremely noisy.

Random walk theory explains exactly this scaling. In the elementary model, a marker starts at 0 on the integer line and moves $+1$ or -1 with equal probability each step. Formally, take independent random variables $Z_1, Z_2, ...$, each $+1$ or -1 with probability $\frac{1}{2}$, set $S_0 = 0$ and $S_n = \sum_{j=1}^{n} Z_j$. The **expected net displacement is zero**, by finite additivity of expectation:

$$E(S_n) = \sum_{j=1}^{n} E(Z_j) = 0.$$

But the **expected squared displacement grows linearly**: using independence (which kills the cross terms) and $E(Z_n^2) = 1$,

$$E(S_n^2) = \sum_{i=1}^{n} E(Z_i^2) + 2 \sum_{1 \le i < j \le n} E(Z_i Z_j) = n.$$

This hints that the expected translation distance after n steps is of order $\sqrt{n}$ — and in fact,

$$\lim_{n \to \infty} \frac{E(|S_n|)}{\sqrt{n}} = \sqrt{\frac{2}{\pi}} \approx 0.8,$$

remarkably close to the constant ratio observed in the simulation. Realizations of such stochastic processes are obtained precisely by Monte Carlo simulation, which is what the code is doing. (The term "random walk" was introduced by Karl Pearson in 1905, and the model is sometimes known as a *drunkard's walk*.)

The masochistic drunk leaves the usual drunk far behind

To compare drunk classes fairly, the experiment first sets `random.seed(0)` so that everyone gets **exactly the same sequence of pseudo-random numbers** — making runs reproducible and comparisons apples-to-apples. Then `simAll` is called on the tuple `(UsualDrunk, MasochistDrunk)` with walk lengths 1000 and 10000, again 100 trials each:

Drunk	Steps	Mean	Max	Min
UsualDrunk	1000	26.828	66.3	4.2
UsualDrunk	10000	90.073	210.6	7.2
MasochistDrunk	1000	58.425	133.3	6.7
MasochistDrunk	10000	515.575	694.6	377.7

The `UsualDrunk` numbers look much like the earlier run — consistent square-root behavior. The `MasochistDrunk` is a different animal:

- At 1000 steps his mean distance (58.425) is **more than double** the usual drunk's.
- At 10000 steps his mean is 515.575 — and his **minimum is 377.7**. Think about that: even the *least successful* masochist ended up almost 378 units from home, farther than the *best* usual-drunk trial at the same length (max 210.6). Every single masochist walk outran every single usual-drunk walk.
- His growth is nearly linear rather than square-root: multiplying steps by ten multiplied his mean by $515.575/58.425 \approx 8.8$, close to ten.

This makes sense mechanically: since the masochist **always moves to a spot farther from where he started**, his distance from home grows monotonically and much faster than a walker whose direction is chosen uniformly at random.

Why plot: turning tables into trends

Tables of numbers are all well and good, but trends like "distance grows as a function of steps" are much easier to see in a picture. The plan for visualizing the trend:

1. Simulate walks of **multiple lengths** for each kind of drunk.
2. Plot the **final distance at the end of each length** of walk for each kind of drunk.
3. Compare the resulting curves directly to see how final distance grows with the number of steps.

Pylab: one interface over a stack of scientific libraries

For the plotting we use Pylab — which isn't really a single thing, but is built on top of several libraries:

- **NumPy** adds vectors, matrices, and many high-level mathematical functions.

- **SciPy** adds mathematical classes and functions useful to scientists.
- **MatPlotLib** adds an object-oriented API for plotting.
- **PyLab** combines the other libraries to provide a MATLAB®-like interface.

Matplotlib itself (a portmanteau of *MATLAB*, *plot*, and *library*) provides an object-oriented API for embedding plots into applications using GUI toolkits like Tkinter, wxPython, Qt, or GTK; there is also a procedural "pylab" interface based on a state machine, designed to closely resemble MATLAB. Originally written by John D. Hunter and distributed under a BSD-style license, it is now a NumFOCUS fiscally sponsored project, and it underpins plotting across the scientific Python ecosystem — pandas uses it as its default backend, and the Event Horizon Telescope collaboration used it while producing the first image of a black hole. Its integration with Jupyter Notebook, allowing inline plots during interactive exploration, makes it a staple of programming and data-visualization education:

Using `pylab.plot`

Three mechanical facts govern `pylab.plot`:

- **The first two arguments must be sequences of the same length.** The first gives the x-coordinates, the second the y-coordinates.
- There are many **optional arguments** controlling how things look.
- **Points are plotted in order**: in the default style, as each point is plotted, a line is drawn connecting it to the previous point. The order of your data therefore matters for what the plot looks like.

A concrete example pulls this together:

```
import pylab
xVals = [1, 2, 3, 4]           # shared x-coordinates
yVals1 = [1, 2, 3, 4]
pylab.plot(xVals, yVals1, 'b-', label = 'first')    # blue solid line
yVals2 = [1, 7, 3, 5]
pylab.plot(xVals, yVals2, 'r--', label = 'second') # red dashed line
pylab.legend()
```

The format string selects style: `'b-'` means a **blue solid line**, `'r--'` a **red dashed line**. Reading the resulting figure teaches two things:

1. **Calling `plot` twice overlays both curves on the same figure** — the blue solid line and the red dashed line are drawn together, which is exactly what lets us later put several drunks' growth curves on one axes for comparison.
2. Because each curve was given a `label` and `pylab.legend()` was called, a **key box appears (in the upper right)** identifying the blue solid line as "first" and the red dashed line as "second".

The data themselves show the ordered-line-segment behavior: `yVals1` traces the straight diagonal $y = x$, while `yVals2` jumps to 7 at $x = 2$, drops to 3, then rises to 5 — each point connected to its predecessor in list order.